

PYTHON TRONG THIẾT KẾ MẠCH SỐ

TỪ TÍNH TOÁN → MÔ PHỎNG LOGIC → TỐI ƯU HÀM BOOLEAN → SINH MÃ PHẦN CỨNG

1. TÍNH TOÁN & XỬ LÝ DỮ LIỆU

Python giúp tính toán nhanh chóng, xử lý dữ liệu đầu vào cho thiết kế mạch.

```
import numpy as np

a = np.array([1, 0, 1, 1])
b = np.array([0, 1, 1, 0])
c = a & b
print("A AND B =", c)

# Kết quả: [0 0 1 0]
```

A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

NumPy

2. MÔ PHỎNG LOGIC

Mô phỏng hoạt động mạch số và kiểm tra tính đúng đắn của thiết kế.

```
from myhdl import *
def xor(a, b, y):
    @always_comb
    def logic():
        y.next = a ^ b
    return logic
```

MyHDL

3. TỐI ƯU HÀM BOOLEAN

Tối thiểu hóa hàm logic để giảm số cổng, giảm độ phức tạp mạch.

Hàm đầu vào:
 $F(A, B, C, D) = \sum m(1, 3, 5, 7, 9, 11, 13, 15)$

CD \ AB	00	01	11	10
00	1	0	1	0
01	0	1	0	1
11	1	1	1	0
10	0	0	1	1

Hàm tối ưu:
 $F = B'D + BD' + AC$

SymPy

4. SINH MÃ PHẦN CỨNG

Tự động sinh mã HDL (Verilog/VHDL) từ mô hình thiết kế.

Verilog

```
module xor_gate(
    input a,
    input b,
    output y
);
    assign y = a ^ b;
endmodule
```

.v

VHDL

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity xor_gate is
    Port ( a : in STD_LOGIC;
          b : in STD_LOGIC;
          y : out STD_LOGIC);
end xor_gate;

architecture Behavioral of
xor_gate is
begin
    y <= a xor b;
end Behavioral;
```

.vhd

Prof Happy AI

Thiết kế

Mô phỏng

Tối ưu

Triển khai

LỢI ÍCH

- ✓ Tăng tốc độ thiết kế và kiểm tra
- ✓ Giảm thiểu sai sót thủ công
- ✓ Tối ưu tài nguyên phần cứng
- ✓ Kết nối liền mạch từ mô hình đến phần cứng

□ Bài giảng về Vibe Coding tạo code để giải quyết các bài toán thiết kế mạch số bằng Python,

Sưu tầm và Biên soạn bổ sung phần Vibe Coding

With the support of Gemini

Prof Happy AI

Date: May 9, 2026

Trong thiết kế mạch số (Digital Circuit Design), việc sử dụng Python không chỉ dừng lại ở việc tính toán mà còn mở rộng sang mô phỏng logic, tối ưu hóa hàm boolean và thậm chí là sinh mã phần cứng (Verilog/VHDL).

Dưới đây là các bài toán điển hình trong thiết kế mạch số mà bạn có thể lập trình bằng Python:

Double-click (or enter) to edit

1. Tối ưu hóa hàm Boolean bằng thuật toán Quine-McCluskey Trong thiết kế mạch, việc rút gọn các biểu thức logic giúp giảm số lượng cổng (gate), từ đó tiết kiệm diện tích chip và năng lượng.

Bài toán: Nhập vào các số minterm, Python sẽ thực hiện rút gọn hàm để tìm biểu thức tối thiểu.

```
# Sử dụng thff viện SymPy để xử lý logic
from sympy import symbols, SOPform
from sympy.logic import simplify_logic

# Định nghĩa các biến logic
A, B, C, D = symbols('A B C D')

# Danh sách các minterm mà tại đó hàm F = 1
minterms = [0, 1, 2, 5, 8, 10, 13]

# Sử dụng SOPform (Sum of Products) để tìm biểu thức tối thiểu
simplified_expr = SOPform([A, B, C, D], minterms)
```

1/17

```
print(f"Hàm Boolean tối thiểu: {simplified_expr}")
```

```
Hàm Boolean tối thiểu: (~B & ~D) | (B & D & ~C) | (~A & ~B & ~C)
```

2. Mô phỏng Bảng sự thật (Truth Table Generation) Để kiểm tra tính đúng đắn của một thiết kế mạch trước khi hiện thực hóa trên phần cứng (FPGA/ASIC).

Bài toán: Viết script Python để tự động tạo bảng sự thật cho một tổ hợp cổng logic phức tạp.

```
import pandas as pd
import itertools

def logic_circuit(A, B, C):
    # Giả sử mạch: Y = (A AND B) OR (NOT C)
    return (A and B) or (not C)

# Tạo tất cả các tổ hợp đầu vào (0, 1) cho 3 biến
inputs = list(itertools.product([0, 1], repeat=3))
results = []

for A, B, C in inputs:
    Y = int(logic_circuit(A, B, C))
    results.append([A, B, C, Y])

# Hiển thị bằng DataFrame cho trực quan
df = pd.DataFrame(results, columns=['A', 'B', 'C', 'Output (Y)'])
print(df.to_string(index=False))
```

A	B	C	Output (Y)
0	0	0	1
0	0	1	0
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	1

3. Thiết kế Bộ đếm (Counters) và Máy trạng thái hữu hạn (FSM) Máy trạng thái là linh hồn của các bộ vi xử lý. Python giúp chúng ta mô phỏng quá trình chuyển trạng thái.

Bài toán: Mô phỏng một bộ đếm nhị phân 3-bit hoặc một máy trạng thái điều khiển đèn giao thông.

```
class SimpleCounter:
    def __init__(self, bits):
        self.limit = 2**bits
        self.current_value = 0

    def tick(self):
        self.current_value = (self.current_value + 1) % self.limit
        return f"{self.current_value:03b}" # Trả về dạng nhị phân

counter = SimpleCounter(3)
print("Kết quả bộ đếm 3-bit:")
for _ in range(8):
    print(counter.tick())
```

Kết quả bộ đếm 3-bit:
001
010
011
100
101
110
111
000

4. Phân tích độ trễ của mạch (Timing Analysis) Trong mạch số cao tốc, độ trễ truyền dẫn (propagation delay) qua các cổng là rất quan trọng để đảm bảo dữ liệu không bị lỗi.

Bài toán: Tính toán tổng độ trễ trên đường dẫn dài nhất (Critical Path).

```
# Định nghĩa độ trễ của các cổng (nanoseconds)
gate_delays = {'AND': 2.0, 'OR': 2.5, 'NOT': 1.0, 'XOR': 3.5}

# Cấu trúc mạch: Input -> NOT -> AND -> OR -> Output
path = ['NOT', 'AND', 'OR']
```

```
total_delay = sum(gate_delays[gate] for gate in path)
print(f"Tổng độ trễ trên đffờng dẫn: {total_delay} ns")
```

Tổng độ trễ trên đffờng dẫn: 5.5 ns

. Sinh mã Verilog tự động (Hardware Generation) Python có thể được dùng làm công cụ Meta-programming để tạo ra hàng nghìn dòng mã phần cứng một cách tự động (đặc biệt hữu ích trong các thiết kế lặp lại như bộ nhớ hoặc ma trận tính toán).

```
def generate_verilog_and_gate(num_inputs):
    inputs = ", ".join([f"in{i}" for i in range(num_inputs)])
    logic = " & ".join([f"in{i}" for i in range(num_inputs)])

    verilog_code = f"""
module and_gate_custom({inputs}, out);
    input {inputs};
    output out;

    assign out = {logic};
endmodule
"""
    return verilog_code

print(generate_verilog_and_gate(4))
```

```
module and_gate_custom(in0, in1, in2, in3, out);
    input in0, in1, in2, in3;
    output out;

    assign out = in0 & in1 & in2 & in3;
endmodule
```

Tại sao nên dùng Python trong thiết kế mạch số? Kiểm thử nhanh: Bạn có thể thử nghiệm thuật toán logic trên Python chỉ trong vài giây trước khi nạp vào chip thật.

Thư viện mạnh mẽ: Các thư viện như PyMTL hoặc MyHDL cho phép bạn thực sự dùng Python để thiết kế và mô phỏng phần cứng như Verilog.

Xử lý dữ liệu: Khi mạch số sinh ra hàng triệu mẫu dữ liệu (ví dụ bộ chuyển đổi ADC), Python là công cụ tốt nhất để phân tích và vẽ đồ thị kết quả.

Double-click (or enter) to edit

Việc mô phỏng một Bộ cộng toàn phần (Full Adder) bằng Python là một bài tập tuyệt vời để hiểu cách các cổng logic kết hợp với nhau nhằm thực hiện phép toán số học. Đây là đơn vị cơ bản nhất trong ALU (Arithmetic Logic Unit) của CPU.

1. Phân tích logic của Full Adder Một bộ Full Adder có 3 đầu vào và 2 đầu ra:

Đầu vào: A , B và Cin (số nhớ từ hàng trước).

Đầu ra: Sum (tổng) và $Cout$ (số nhớ ra hàng sau).

Công thức logic: $Sum = (A \oplus B) \oplus Cin$ $Cout = (A \cdot B) + (Cin \cdot (A \oplus B))$

Mô phỏng bằng hàm Python cơ bản

Cách đơn giản nhất là định nghĩa các cổng logic thành các hàm và kết hợp chúng lại.

```
import pandas as pd

# Định nghĩa các cổng logic cơ bản
def XOR(a, b): return a ^ b
def AND(a, b): return a & b
def OR(a, b): return a | b

# Mô phỏng Full Adder
def full_adder(A, B, Cin):
    # Bffớc 1: Tính toán tầng XOR đầu tiên
    xor1 = XOR(A, B)

    # Bffớc 2: Tính Sum
    sum_out = XOR(xor1, Cin)

    # Bffớc 3: Tính toán tầng AND và OR để ra Cout
    and1 = AND(A, B)
    and2 = AND(xor1, Cin)
    cout = OR(and1, and2)
```

```

        return sum_out, cout

# Tạo bảng sự thật (Truth Table)
results = []
for A in [0, 1]:
    for B in [0, 1]:
        for Cin in [0, 1]:
            S, C = full_adder(A, B, Cin)
            results.append({"A": A, "B": B, "Cin": Cin, "Sum": S, "Cout": C})

df = pd.DataFrame(results)
print("Bảng sự thật của Bộ cộng toàn phần (Full Adder):")
print(df.to_string(index=False))

```

Bảng sự thật của Bộ cộng toàn phần (Full Adder):

A	B	Cin	Sum	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

□ 4. Trực quan hóa kết quả

Chúng ta có thể dùng matplotlib để vẽ biểu đồ thời gian (Timing Diagram) nhằm mô phỏng cách các tín hiệu điện thay đổi.

```

import matplotlib.pyplot as plt

# Giả lập tín hiệu đầu vào thay đổi theo thời gian
time = list(range(8))
A_sig = [0, 1, 0, 1, 0, 1, 0, 1]
B_sig = [0, 0, 1, 1, 0, 0, 1, 1]
Cin_sig = [0, 0, 0, 0, 1, 1, 1, 1]

S_res, C_res = [], []
for i in range(8):
    s, c = full_adder(A_sig[i], B_sig[i], Cin_sig[i])
    S_res.append(s)
    C_res.append(c)

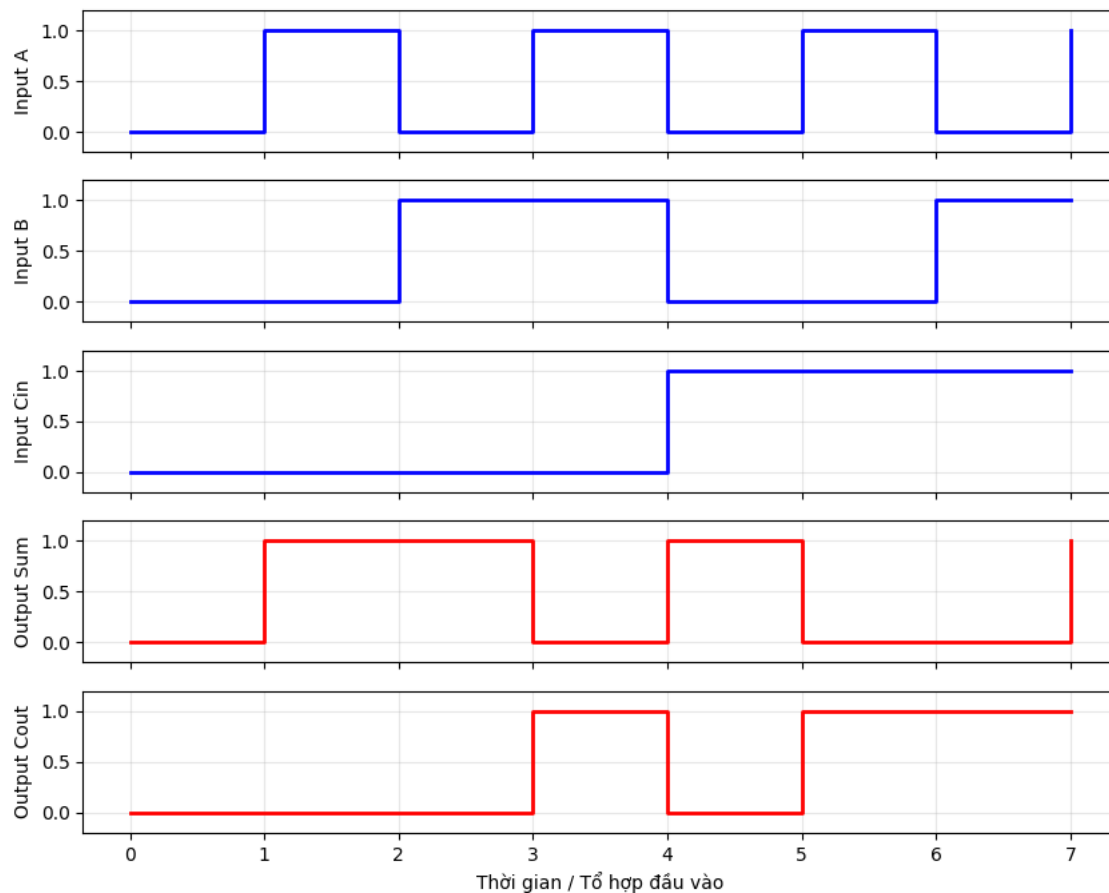
# Vẽ biểu đồ thời gian
fig, axs = plt.subplots(5, 1, figsize=(10, 8), sharex=True)
signals = [A_sig, B_sig, Cin_sig, S_res, C_res]
labels = ['Input A', 'Input B', 'Input Cin', 'Output Sum', 'Output Cout']
colors = ['b', 'b', 'b', 'r', 'r']

for i in range(5):
    axs[i].step(time, signals[i], where='post', color=colors[i], lw=2)
    axs[i].set_ylabel(labels[i])
    axs[i].set_ylim(-0.2, 1.2)
    axs[i].grid(True, alpha=0.3)

plt.xlabel("Thời gian / Tổ hợp đầu vào")
plt.suptitle("Biểu đồ thời gian mô phỏng Full Adder")
plt.show()

```

Biểu đồ thời gian mô phỏng Full Adder



Tại sao mô phỏng này quan trọng?

Kiểm tra thiết kế: Bạn có thể xác nhận logic của mình đúng cho tất cả các trường hợp trước khi viết code Verilog thực sự.

Mở rộng quy mô: Từ một FullAdder, bạn có thể dễ dàng tạo ra một bộ cộng 64-bit chỉ bằng một vòng lặp, điều mà vẽ tay sẽ mất hàng giờ.

Giáo dục: Giúp người học thấy rõ cách các bit "nhớ" (carry) được truyền từ hàng thấp lên hàng cao trong một hệ thống máy tính.

Hãy mô phỏng một bộ ALU đơn giản (có thể cộng, trừ, AND, OR) dựa trên cấu trúc này

□ Việc xây dựng một bộ ALU (Arithmetic Logic Unit) là bước tiếp theo để tiến gần hơn tới kiến trúc của một bộ vi xử lý thực thụ.

Một bộ ALU đơn giản thường nhận hai đầu vào (A, B), một mã điều khiển (Opcode) để quyết định phép toán nào sẽ được thực hiện. Dưới đây là mã Python mô phỏng bộ ALU 4-bit, tích hợp các phép toán số học và logic:

```
import pandas as pd

class SimpleALU:
    def __init__(self):
        # Định nghĩa các mã Opcode (Mã lệnh)
        self.OP_ADD = 0 # 00 in binary
        self.OP_SUB = 1 # 01 in binary
        self.OP_AND = 2 # 10 in binary
        self.OP_OR = 3 # 11 in binary

    def _to_binary_list(self, val, bits=4):
        """Chuyển số nguyên thành danh sách các bit"""
        return [int(x) for x in bin(val & (2**bits - 1))[2:].zfill(bits)]

    def execute(self, A, B, opcode):
        """Thực hiện phép toán dựa trên Opcode"""
        # Đảm bảo đầu vào chỉ nằm trong phạm vi 4-bit (0-15)
        A = A & 0xF
```

```
B = B & 0xF

result = 0
status = {"Zero": 0, "Negative": 0}

if opcode == self.OP_ADD:
    result = A + B
elif opcode == self.OP_SUB:
    result = A - B
elif opcode == self.OP_AND:
    result = A & B
elif opcode == self.OP_OR:
    result = A | B

# Xử lý kết quả trả về trong phạm vi 4-bit
final_result = result & 0xF

# Cập nhật các cờ trạng thái (Flags)
status["Zero"] = 1 if final_result == 0 else 0
status["Negative"] = 1 if (final_result & 0x8) else 0 # Kiểm tra bit MSB (bit dấu)

return final_result, status

# 2. Chạy thử nghiệm bộ ALU
alu = SimpleALU()
test_cases = [
    (5, 3, alu.OP_ADD), # 5 + 3 = 8
    (10, 4, alu.OP_SUB), # 10 - 4 = 6
    (12, 10, alu.OP_AND), # 1100 & 1010 = 1000 (8)
    (5, 10, alu.OP_OR) # 0101 | 1010 = 1111 (15)
]

data = []
op_names = {0: "ADD", 1: "SUB", 2: "AND", 3: "OR"}

for a, b, op in test_cases:
    res, flags = alu.execute(a, b, op)
    data.append({
        "Phép toán": op_names[op],
        "A (Dec)": a, "B (Dec)": b,
        "Kết quả (Dec)": res,
        "Kết quả (Bin)": bin(res)[2:].zfill(4),
        "Cờ Zero": flags["Zero"],
        "Cờ Neg": flags["Negative"]
    })

print(pd.DataFrame(data).to_string(index=False))
```

Phép toán	A (Dec)	B (Dec)	Kết quả (Dec)	Kết quả (Bin)	Cờ Zero	Cờ Neg
ADD	5	3	8	1000	0	1
SUB	10	4	6	0110	0	0
AND	12	10	8	1000	0	1
OR	5	10	15	1111	0	1

□ Giải thích cấu trúc mô phỏng

Opcode (Mã lệnh): Đây là đầu vào điều khiển. Trong thực tế, bộ giải mã lệnh (Instruction Decoder) sẽ gửi tín hiệu này đến ALU để chọn đường dẫn dữ liệu.

Xử lý bitwise: Các phép toán & 0xF giúp giới hạn dữ liệu trong 4-bit, mô phỏng đúng hành vi của các thanh ghi (registers) phần cứng.

Cờ trạng thái (Flags):

Zero Flag: Rất quan trọng cho các lệnh rẽ nhánh (như if trong lập trình).

Negative Flag: Xác định kết quả có phải là số âm hay không dựa trên bit trọng số cao nhất (MSB).

Để trực quan hóa đường đi của dữ liệu trong ALU, chúng ta cần mô phỏng cách một bộ Multiplexer (MUX) hoạt động như một "ngã tư" điều hướng dữ liệu. Tùy thuộc vào Opcode, MUX sẽ chọn tín hiệu từ các khối chức năng (ADD, SUB, AND, OR) để đưa ra ngõ ra cuối cùng.

Dưới đây là đoạn code sử dụng matplotlib để vẽ sơ đồ khối và làm nổi bật đường đi của dữ liệu dựa trên mã lệnh:

1. Python Code: Trực quan hóa sơ đồ khối ALU

```
import matplotlib.pyplot as plt
import matplotlib.patches as patches

def visualize_alu_path(opcode):
    fig, ax = plt.subplots(figsize=(12, 8))
```

```

# 1. Vẽ các khối chức năng (Functional Units)
units = {
    0: "ADDER",
    1: "SUBTRACTOR",
    2: "AND GATE",
    3: "OR GATE"
}

unit_boxes = {}
for i, name in units.items():
    # Vẽ hình chữ nhật cho mỗi khối
    box = patches.Rectangle((3, 6 - i*1.5), 2, 1, linewidth=2, edgecolor='black', facecolor='white')
    ax.add_patch(box)
    ax.text(4, 6.5 - i*1.5, name, ha='center', va='center', fontweight='bold')
    unit_boxes[i] = (5, 6.5 - i*1.5) # Điểm ra của khối

# 2. Vẽ bộ Multiplexer (MUX) - Hình thang
mux_coords = [[7, 6.5], [8, 5.5], [8, 2.5], [7, 1.5]]
mux = patches.Polygon(mux_coords, closed=True, linewidth=2, edgecolor='black', facecolor='f0f0f0')
ax.add_patch(mux)
ax.text(7.5, 4, "MUX\n(Selector)", ha='center', va='center', rotation=270, fontweight='bold')

# 3. Vẽ đường dẫn dữ liệu (Bus)
# Đường vào A và B
ax.annotate('Input A', xy=(3, 6.5), xytext=(1, 4), arrowprops=dict(arrowstyle="->", connectionstyle="angle,angleA=0,angleB=0"),
ax.annotate('Input B', xy=(3, 6.4), xytext=(1, 3.8), arrowprops=dict(arrowstyle="->", connectionstyle="angle,angleA=0,angleB=0"),

# Vẽ đường từ các khối vào MUX
for i in range(4):
    color = 'red' if i == opcode else 'gray'
    alpha = 1.0 if i == opcode else 0.3
    width = 3 if i == opcode else 1

    # Đường từ khối đến MUX
    y_pos = 6.5 - i*1.5
    ax.plot([5, 7], [y_pos, y_pos], color=color, lw=width, alpha=alpha)

    if i == opcode:
        ax.text(6, y_pos + 0.2, "ACTIVE PATH", color='red', fontsize=9, fontweight='bold')

# 4. Tín hiệu điều khiển Opcode (Control Signal)
ax.annotate(f'Opcode: {opcode}', xy=(7.5, 2.3), xytext=(7.5, 0.5),
          arrowprops=dict(facecolor='blue', shrink=0.05), ha='center', color='blue', fontweight='bold')

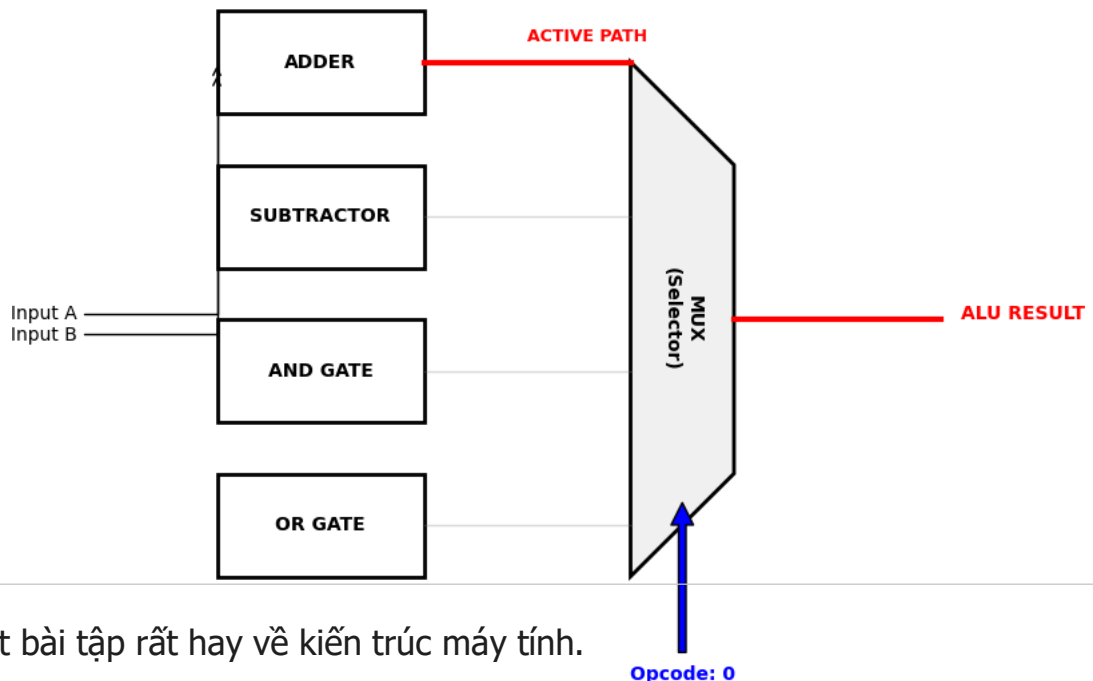
# 5. Đường ra cuối cùng (Output)
ax.plot([8, 10], [4, 4], color='red', lw=3)
ax.text(10.2, 4, "ALU RESULT", fontweight='bold', color='red')

# Thiết lập khung nhìn
ax.set_xlim(0, 12)
ax.set_ylim(0, 8)
ax.axis('off')
plt.title(f"Mô phỏng đường đi dữ liệu trong ALU - Đang thực hiện: {units[opcode]}", fontsize=15)
plt.show()

# Thử nghiệm với các Opcode khác nhau
# 0: ADD, 1: SUB, 2: AND, 3: OR
visualize_alu_path(opcode=0)

```

Mô phỏng đường đi dữ liệu trong ALU - Đang thực hiện: ADDER



□ Đây là một bài tập rất hay về kiến trúc máy tính.

Trong phần cứng thực tế, việc sử dụng bộ cộng để thực hiện phép trừ giúp tiết kiệm đáng kể diện tích chip vì ta không cần thiết kế một bộ trừ riêng biệt.

Nguyên lý cốt lõi là: $A - B = A + (-B)$.

Trong hệ nhị phân, $-B$ được biểu diễn bằng Số bù 2, tính bằng cách lấy đảo bit của B (bù 1) rồi cộng thêm 1. Python Code: Mô phỏng ALU thực hiện phép trừ qua Bộ cộng

```
import pandas as pd

class HardwareALU:
    def __init__(self, bits=4):
        self.bits = bits
        self.mask = (1 << bits) - 1 # 0xF cho 4-bit

    def _full_adder_logic(self, a_bits, b_bits, cin):
        """Mô phỏng logic của bộ cộng Ripple Carry Adder tầng thấp"""
        res_bits = []
        carry = cin
        # Duyệt từ bit thấp nhất (LSB) đến cao nhất (MSB)
        for i in range(self.bits):
            a = a_bits[i]
            b = b_bits[i]

            # Công thức Full Adder
            sum_bit = a ^ b ^ carry
            carry = (a & b) | (carry & (a ^ b))
            res_bits.append(sum_bit)

        return res_bits, carry

    def execute(self, A, B, mode):
        """
        mode='ADD': A + B
        mode='SUB': A - B (Thực chất là A + NOT(B) + 1)
        """
        # 1. Chuyển số nguyên thành danh sách bit (đảo ngược để bit 0 ở chỉ số 0)
        a_bits = [(A >> i) & 1 for i in range(self.bits)]
        b_bits = [(B >> i) & 1 for i in range(self.bits)]

        cin = 0
        if mode == 'SUB':
            # BƯỚC A: Lấy bù 1 (Đảo ngược tất cả các bit của B)
            b_bits = [1 - b for b in b_bits]
            # BƯỚC B: Để lấy bù 2, ta cộng thêm 1 bằng cách đặt Carry-in ban đầu = 1
            cin = 1
```



```
# BƯỚC C: Thực hiện phép cộng thông qua bộ cộng duy nhất
res_bits, cout = self._full_adder_logic(a_bits, b_bits, cin)

# Chuyển danh sách bit ngược lại thành số nguyên
result = sum(bit << i for i, bit in enumerate(res_bits))

# Kiểm tra cờ trạng thái
is_zero = 1 if result == 0 else 0
is_neg = 1 if res_bits[-1] == 1 else 0 # Kiểm tra bit dấu (MSB)

return result, {"Cout": cout, "Zero": is_zero, "Neg": is_neg}

# --- CHẠY THỬ NGHIỆM ---
alu = HardwareALU(bits=4)
tests = [
    (5, 3, 'ADD'), # 5 + 3 = 8
    (10, 4, 'SUB'), # 10 - 4 = 6
    (3, 7, 'SUB'), # 3 - 7 = -4 (Trong 4-bit bù 2, -4 là 1100 -> 12 Dec)
]

results = []
for a, b, mode in tests:
    res, flags = alu.execute(a, b, mode)
    results.append({
        "Phép toán": f"{a} {mode} {b}",
        "Kết quả (Dec)": res,
        "Kết quả (Bin)": bin(res)[2:].zfill(4),
        "Cờ Nhớ (Cout)": flags["Cout"],
        "Cờ Âm (Neg)": flags["Neg"]
    })

print(pd.DataFrame(results).to_string(index=False))
```

Phép toán	Kết quả (Dec)	Kết quả (Bin)	Cờ Nhớ (Cout)	Cờ Âm (Neg)
5 ADD 3	8	1000	0	1
10 SUB 4	6	0110	1	0
3 SUB 7	12	1100	0	1

Trong kiến trúc máy tính, phép nhân (Multiplication) thường được thực hiện thông qua thuật toán "Dịch và Cộng" (Shift and Add). Thay vì dùng một bộ nhân khổng lồ, ALU sẽ lặp lại việc kiểm tra từng bit của số nhân và cộng dồn số bị nhân vào kết quả sau mỗi lần dịch chuyển.

Dưới đây là mã Python mô phỏng bộ nhân nhị phân 4-bit theo đúng nguyên lý phần cứng:

```
import pandas as pd

class MultiplierALU:
    def __init__(self, bits=4):
        self.bits = bits
        # Kết quả của phép nhân n-bit có thể lên tới 2n-bit
        self.result_mask = (1 << (2 * bits)) - 1

    def multiply(self, A, B):
        """
        Mô phỏng thuật toán Shift and Add:
        A: Multiplicand (Số bị nhân)
        B: Multiplier (Số nhân)
        """
        # Đảm bảo đầu vào là n-bit
        multiplicand = A & ((1 << self.bits) - 1)
        multiplier = B & ((1 << self.bits) - 1)

        accumulator = 0 # Thanh tích lũy kết quả
        steps = []

        # Duyệt qua từng bit của số nhân B (từ LSB đến MSB)
        for i in range(self.bits):
            # Kiểm tra bit thứ i của B có bằng 1 không
            bit_is_one = (multiplier >> i) & 1

            if bit_is_one:
                # Nếu bit là 1, cộng số bị nhân đã dịch trái vào accumulator
                term = multiplicand << i
                accumulator += term
                action = f"Cộng (A << {i})"
            else:
                term = 0
                action = "Bỏ qua (Bit=0)"

            steps.append({
                "Bffở": i,

```

```

        "Bit của B": bit_is_one,
        "Giá trị cộng": bin(term)[2:].zfill(2 * self.bits),
        "Hành động": action,
        "Tích lũy": bin(accumulator & self.result_mask)[2:].zfill(2 * self.bits)
    })

    return accumulator & self.result_mask, steps

# --- CHẠY MÔ PHÒNG ---
alu_mul = MultiplierALU(bits=4)
A, B = 13, 5 # 13 (1101) * 5 (0101) = 65 (01000001)

result, process = alu_mul.multiply(A, B)

print(f"--- MÔ PHÒNG PHÉP NHÂN: {A} x {B} ---")
print(pd.DataFrame(process).to_string(index=False))
print(f"\nKết quả cuối cùng: {result} (Nhị phân: {bin(result)[2:].zfill(8)})")

```

```

--- MÔ PHÒNG PHÉP NHÂN: 13 x 5 ---
Bffốc  Bit của B  Giá trị cộng      Hành động  Tích lũy
0       1         00001101  Cộng (A << 0) 00001101
1       0         00000000  Bỏ qua (Bit=0) 00001101
2       1         00110100  Cộng (A << 2) 01000001
3       0         00000000  Bỏ qua (Bit=0) 01000001

```

Kết quả cuối cùng: 65 (Nhị phân: 01000001)

2. Giải thích nguyên lý hoạt độngThuật toán này mô phỏng chính xác cách chúng ta nhân tay trong hệ thập phân, nhưng áp dụng cho hệ nhị phân:Kiểm tra từng bit (Scanning): ALU kiểm tra từng bit của số nhân B từ phải sang trái.Quyết định cộng (Conditional Add):Nếu bit của B là 1: Lấy số bị nhân A , dịch trái tương ứng với vị trí của bit đó và cộng vào tổng tạm thời.Nếu bit của B là 0: Không làm gì cả (cộng 0).Dịch trái (Shifting): Mỗi lần dịch trái một vị trí tương đương với việc nhân số bị nhân A với 2^i .3. Tại sao kết quả cần gấp đôi số bit?Trong đại số tuyến tính và mạch số, khi bạn nhân hai số n -bit, giá trị cực đại có thể đạt được là:

$$(2^n - 1) \times (2^n - 1) \approx 2^{2n}$$

Ví dụ: Với mạch 4-bit, số lớn nhất là 15. $15 \times 15 = 225$. Để biểu diễn số 225, ta cần ít nhất 8 bit. Đây là lý do tại sao các thanh ghi kết quả của phép nhân trong CPU (như cặp thanh ghi DX:AX trong kiến trúc x86) thường có kích thước gấp đôi thanh ghi thông thường.4. Ưu và nhược điểm của phương pháp nàyƯu điểm: Cấu trúc cực kỳ đơn giản, chỉ cần tận dụng bộ Adder và bộ Shifter đã có sẵn trong ALU.Nhược điểm: Tốc độ chậm. Với mạch n -bit, ta mất n chu kỳ clock để hoàn thành.

Mở rộng: Trong các chip hiện đại, người ta sử dụng Wallace Tree hoặc Booth's Algorithm để thực hiện phép nhân nhanh hơn bằng cách cộng song song nhiều bit cùng lúc thay vì đợi từng bước như trên. Bạn có muốn thử tìm hiểu cách thuật toán Booth xử lý số nhân có dấu không?

Thuật toán Booth (Booth's Algorithm) là một bước tiến lớn trong thiết kế ALU, vì nó cho phép nhân các số nguyên có dấu (số bù 2) một cách trực tiếp mà không cần chuyển đổi về số dương.

Thay vì kiểm tra từng bit đơn lẻ, thuật toán Booth kiểm tra cặp bit để quyết định thực hiện phép cộng, phép trừ hoặc chỉ dịch bit. Điều này giúp giảm số lượng phép cộng cần thiết khi gặp các dãy số 1 liên tiếp.

- Nguyên lý thuật toán BoothThuật toán dựa trên việc quan sát cặp bit *Current_Bit* và *Previous_Bit* (ban đầu *Previous_Bit* = 0):10: Bắt đầu chuỗi số 1 → Thực hiện Trừ: $Accumulator = Accumulator - A$.01: Kết thúc chuỗi số 1 → Thực hiện Cộng: $Accumulator = Accumulator + A$.00 hoặc 11: Bên trong chuỗi số 0 hoặc số 1 → Chỉ Dịch phải số học.
- Python Code: Mô phỏng thuật toán Booth 4-bit

```

import pandas as pd

class BoothMultiplier:
    def __init__(self, bits=4):
        self.bits = bits

    def _arithmetic_right_shift(self, A, Q, Q_prev):
        """Dịch phải số học cho thành ghi kết hợp [A, Q, Q_prev]"""
        # Kết hợp A và Q thành một chuỗi bit duy nhất
        combined = (A << (self.bits + 1)) | (Q << 1) | Q_prev

        # Dịch phải 1 bit
        combined >>= 1

        # Giữ nguyên bit dấu (MSB của A)
        msb = (combined >> (2 * self.bits)) & 1

```

```

        if (A >> (self.bits - 1)) & 1:
            combined |= (1 << (2 * self.bits))

# Tách lại các giá trị
new_q_prev = combined & 1
new_q = (combined >> 1) & ((1 << self.bits) - 1)
new_a = (combined >> (self.bits + 1)) & ((1 << self.bits) - 1)

return new_a, new_q, new_q_prev

def multiply(self, M, Q):
    """
    M: Multiplicand (Số bị nhân)
    Q: Multiplier (Số nhân)
    """
    A = 0          # Accumulator
    Q_prev = 0     # Bit phụ phía sau LSB của Q
    m = M & ((1 << self.bits) - 1)
    q = Q & ((1 << self.bits) - 1)

    steps = []

    for i in range(self.bits):
        q_lsb = q & 1
        action = ""

        # Kiểm tra cặp bit (Q_lsb, Q_prev)
        if q_lsb == 1 and Q_prev == 0:
            A = (A - m) & ((1 << self.bits) - 1)
            action = "A = A - M"
        elif q_lsb == 0 and Q_prev == 1:
            A = (A + m) & ((1 << self.bits) - 1)
            action = "A = A + M"
        else:
            action = "Chỉ dịch bit"

        # Dịch phải số học (Arithmetic Shift Right)
        A, q, Q_prev = self._arithmetic_right_shift(A, q, Q_prev)

        steps.append({
            "Vòng": i + 1,
            "Cặp bit": f"{q_lsb}{Q_prev}",
            "Thao tác": action,
            "A": bin(A)[2:].zfill(self.bits),
            "Q": bin(q)[2:].zfill(self.bits),
            "Q_{-1}": Q_prev
        })

    # Kết quả cuối cùng là sự kết hợp của A và Q
    final_res = (A << self.bits) | q
    # Xử lý số âm nếu MSB là 1 (chuyển từ bù 2 sang giá trị thực)
    if (A >> (self.bits - 1)) & 1:
        final_res -= (1 << (2 * self.bits))

    return final_res, steps

# --- CHẠY THỬ NGHIỆM ---
booth = BoothMultiplier(4)
multiplicand = 3    # 0011
multiplier = -4     # 1100 (Bù 2)

result, process = booth.multiply(multiplicand, multiplier)

print(f"--- THUẬT TOÁN BOOTH: {multiplicand} x {multiplier} ---")
print(pd.DataFrame(process).to_string(index=False))
print(f"\nKết quả cuối cùng: {result}")

```

```

--- THUẬT TOÁN BOOTH: 3 x -4 ---
Vòng  Cặp bit   Thao tác   A      Q  Q_{-1}
1      00  Chỉ dịch bit  0000  0110    0
2      00  Chỉ dịch bit  0000  0011    0
3      11    A = A - M  1110  1001    1
4      11  Chỉ dịch bit  1111  0100    1

```

Kết quả cuối cùng: -12

Để trực quan hóa thuật toán Booth, chúng ta có thể sử dụng thư viện matplotlib để tạo ra một bảng trạng thái động hoặc sơ đồ mô phỏng các thanh ghi qua từng bước.

Dưới đây là đoạn code bổ sung giúp bạn vẽ lại quá trình thay đổi của các thanh ghi A, Q và Q_{-1} . Điều này giúp quan sát rõ ràng cách dữ liệu "dịch" dần từ trái sang phải.

1. Python Code: Trực quan hóa các bước của thuật toán Booth

```
import matplotlib.pyplot as plt

def visualize_booth_process(multiplicand, multiplier, steps):
    fig, ax = plt.subplots(figsize=(12, 6))
    ax.set_axis_off()

    # Thiết lập tiêu đề và thông tin chung
    plt.title(f"Trực quan hóa thuật toán Booth: {multiplicand} x {multiplier}",
              fontsize=16, fontweight='bold', pad=20)

    # Tạo bảng dữ liệu trực quan
    header = ["Vòng", "Thao tác", "Thanh ghi A", "Thanh ghi Q", "Q_-1", "Kết hợp (A:Q:Q_-1)"]
    rows = []
    for s in steps:
        combined = f"{s['A']} : {s['Q']} : {s['Q_-1']}"
        rows.append([s['Vòng'], s['Thao tác'], s['A'], s['Q'], s['Q_-1'], combined])

    # Vẽ bảng bằng Matplotlib
    table = ax.table(cellText=rows, colLabels=header, loc='center', cellLoc='center')
    table.auto_set_font_size(False)
    table.set_fontsize(11)
    table.scale(1.2, 2.5)

    # Tô màu các dòng để dễ theo dõi
    for i, key in enumerate(table.get_celld().keys()):
        cell = table.get_celld()[key]
        if key[0] == 0: # Header
            cell.set_facecolor('#40466e')
            cell.get_text().set_color('white')
        elif key[0] % 2 == 0:
            cell.set_facecolor('#f2f2f2')

    plt.show()

# --- SỬ DỤNG LẠI HÀM MULTIPLY TỪ CODE CỦA BẠN ---
booth = BoothMultiplier(4)
m, q_val = 3, -4
result, process = booth.multiply(m, q_val)

# Gọi hàm trực quan hóa
visualize_booth_process(m, q_val, process)
```

Trực quan hóa thuật toán Booth: 3 x -4

Vòng	Thao tác	Thanh ghi A	Thanh ghi Q	Q_-1	Kết hợp (A:Q:Q_-1)
1	Chỉ dịch bit	0000	0110	0	0000 : 0110 : 0
2	Chỉ dịch bit	0000	0011	0	0000 : 0011 : 0
3	A = A - M	1110	1001	1	1110 : 1001 : 1
4	Chỉ dịch bit	1111	0100	1	1111 : 0100 : 1

Để tạo thành một bộ vi xử lý (CPU) hoàn chỉnh, chúng ta cần kết hợp khối ALU với Thanh ghi (Registers) để lưu trữ dữ liệu tạm thời và Bộ nhớ (Memory) để lưu trữ chương trình và dữ liệu dài hạn. Tất cả được điều khiển bởi một Đơn vị điều khiển (Control Unit) theo chu trình: Fetch - Decode - Execute (Nhận lệnh - Giải mã - Thực thi).

Dưới đây là mã mô phỏng một máy tính đơn giản (Toy CPU) 8-bit:

1. Python Code: Mô phỏng CPU hoàn chỉnh

```

import pandas as pd

class SimpleCPU:
    def __init__(self, memory_size=16):
        # 1. Bộ nhớ (RAM)
        self.memory = [0] * memory_size

        # 2. Các thanh ghi (Registers)
        self.R1 = 0 # Thanh ghi đa năng 1
        self.R2 = 0 # Thanh ghi đa năng 2
        self.PC = 0 # Program Counter (Trình đếm chffơng trình)
        self.IR = 0 # Instruction Register (Thanh ghi lệnh)

        # 3. Tập lệnh (Instruction Set)
        self.LOAD = 0b0001 # Load từ RAM vào R1
        self.STORE = 0b0010 # Store từ R1 vào RAM
        self.ADD = 0b0011 # R1 = R1 + R2
        self.HALT = 0b1111 # Dừng chffơng trình

    def fetch_decode_execute(self):
        log = []
        running = True

        while running:
            # --- FETCH ---
            self.IR = self.memory[self.PC]
            current_pc = self.PC
            self.PC += 1

            # --- DECODE ---
            opcode = (self.IR >> 4) & 0xF # 4 bit đầu là mã lệnh
            operand = self.IR & 0xF # 4 bit sau là địa chỉ hoặc dữ liệu

            action = ""
            # --- EXECUTE ---
            if opcode == self.LOAD:
                self.R1 = self.memory[operand]
                action = f"LOAD R1 từ địa chỉ {operand}"
            elif opcode == self.STORE:
                self.memory[operand] = self.R1
                action = f"STORE R1 vào địa chỉ {operand}"
            elif opcode == self.ADD:
                old_r1 = self.R1
                self.R1 = (self.R1 + self.R2) & 0xFF # ALU thực hiện phép cộng
                action = f"ALU ADD: {old_r1} + {self.R2} = {self.R1}"
            elif opcode == self.HALT:
                action = "HALT (Dừng)"
                running = False

            log.append({
                "PC": current_pc,
                "Lệnh": bin(self.IR)[2:].zfill(8),
                "Hành động": action,
                "R1": self.R1,
                "R2": self.R2,
                "RAM[15]": self.memory[15] # Theo dõi ô nhớ kết quả
            })

        return log

# --- CHƯƠNG TRÌNH MẪU ---
# Bài toán: Lấy số ở ô nhớ 14, cộng với R2, lffu kết quả vào ô nhớ 15
cpu = SimpleCPU()

# Nạp dữ liệu vào RAM
cpu.memory[14] = 10 # Dữ liệu đầu vào
cpu.R2 = 5 # Giả sử R2 đã có sẵn giá trị 5
cpu.memory[0] = (cpu.LOAD << 4) | 14 # Lệnh: LOAD R1, [14]
cpu.memory[1] = (cpu.ADD << 4) | 0 # Lệnh: ADD (R1 = R1 + R2)
cpu.memory[2] = (cpu.STORE << 4) | 15 # Lệnh: STORE [15], R1
cpu.memory[3] = (cpu.HALT << 4) | 0 # Lệnh: HALT

# Chạy CPU
history = cpu.fetch_decode_execute()

print("---- QUÁ TRÌNH THỰC THI CỦA CPU ----")
print(pd.DataFrame(history).to_string(index=False))

```

```

--- QUÁ TRÌNH THỰC THI CỦA CPU ---
PC   Lệnh           Hành động   R1  R2  RAM[15]
0 00011110  LOAD R1 từ địa chỉ 14  10  5   0
1 00110000  ALU ADD: 10 + 5 = 15  15  5   0

```

2	00101111	STORE R1	vào địa chỉ	15	15	5	15
3	11110000	HALT (Dừng)		15	5		15

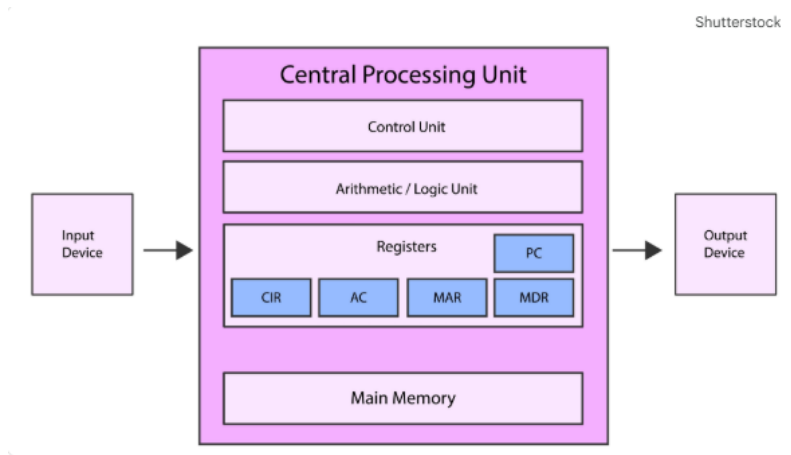
2. Sự kết hợp giữa các khối

Để một CPU hoạt động, các khối này tương tác với nhau thông qua hệ thống Bus:

Program Counter (PC): Đóng vai trò là "ngón tay" chỉ vào dòng lệnh tiếp theo trong bộ nhớ.

ALU & Registers: Khi lệnh ADD được giải mã, Control Unit sẽ mở đường truyền dữ liệu từ các thanh ghi (R1, R2) vào đầu vào của ALU, sau đó lấy kết quả từ đầu ra ALU ghi đè lại vào thanh ghi.

Memory Interface: Lệnh LOAD và STORE kích hoạt các đường dây địa chỉ (Address Bus) để di chuyển dữ liệu giữa RAM và các thanh ghi bên trong CPU.



3. Ý nghĩa của mô phỏng hoàn chỉnh

Kiến trúc Von Neumann: Mô phỏng này cho thấy chương trình và dữ liệu cùng nằm chung trong một bộ nhớ (RAM), đây là nền tảng của hầu hết máy tính hiện nay.

Chu kỳ máy (Clock Cycle): Mỗi bước trong vòng lặp while tương ứng với các nhịp đồng hồ của máy tính. CPU càng nhanh thì chu kỳ này càng ngắn.

Tập lệnh (Instruction Set): Bạn có thể thấy cách chúng ta định nghĩa các con số nhị phân (0b0001, 0b0011) để điều khiển phần cứng. Đây chính là Ngôn ngữ máy (Machine Code).

4. Kết luận Khi kết hợp tất cả lại, chúng ta không còn nhìn thấy các cổng logic rời rạc, mà nhìn thấy một hệ thống có khả năng tính toán tự động.

ALU là cơ bắp (tính toán).

Thanh ghi là trí nhớ ngắn hạn (tốc độ cao).

Bộ nhớ là thư viện (lưu trữ).

Control Unit là bộ não (điều phối).

Lập trình mô phỏng này giúp các kỹ sư hiểu rõ cách phần mềm "nói chuyện" với phần cứng, từ đó tối ưu hóa mã nguồn và thiết kế những bộ vi xử lý mạnh mẽ hơn trong tương lai.

- Để trực quan hóa quá trình thực thi của một CPU (Central Processing Unit), cách tốt nhất là theo dõi sự thay đổi trạng thái của các Thanh ghi (Registers) và Bộ nhớ (RAM) qua từng chu kỳ lệnh.

Dưới đây là mã Python sử dụng matplotlib để vẽ biểu đồ diễn biến của dữ liệu bên trong CPU, giúp bạn hình dung cách lệnh được "chạy" qua các khối chức năng.

1. Python Code: Trực quan hóa chu kỳ thực thi CPU

```
import matplotlib.pyplot as plt
import pandas as pd

# Giả sử history là kết quả từ hàm fetch_decode_execute() của bạn
# history = cpu.fetch_decode_execute()
```

```
def visualize_cpu_execution(history):
    df = pd.DataFrame(history)
    steps = range(len(df))

    fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(12, 10), sharex=True)

    # --- BIỂU ĐỒ 1: TRẠNG THÁI THANH GHI ---
    ax1.plot(steps, df['R1'], marker='o', label='Thanh ghi R1', color='blue', linewidth=2)
    ax1.plot(steps, df['R2'], marker='s', label='Thanh ghi R2', color='green', linestyle='--')
    ax1.plot(steps, df['RAM[15]'], marker='D', label='RAM tại ô nhớ [15]', color='red', linewidth=2)

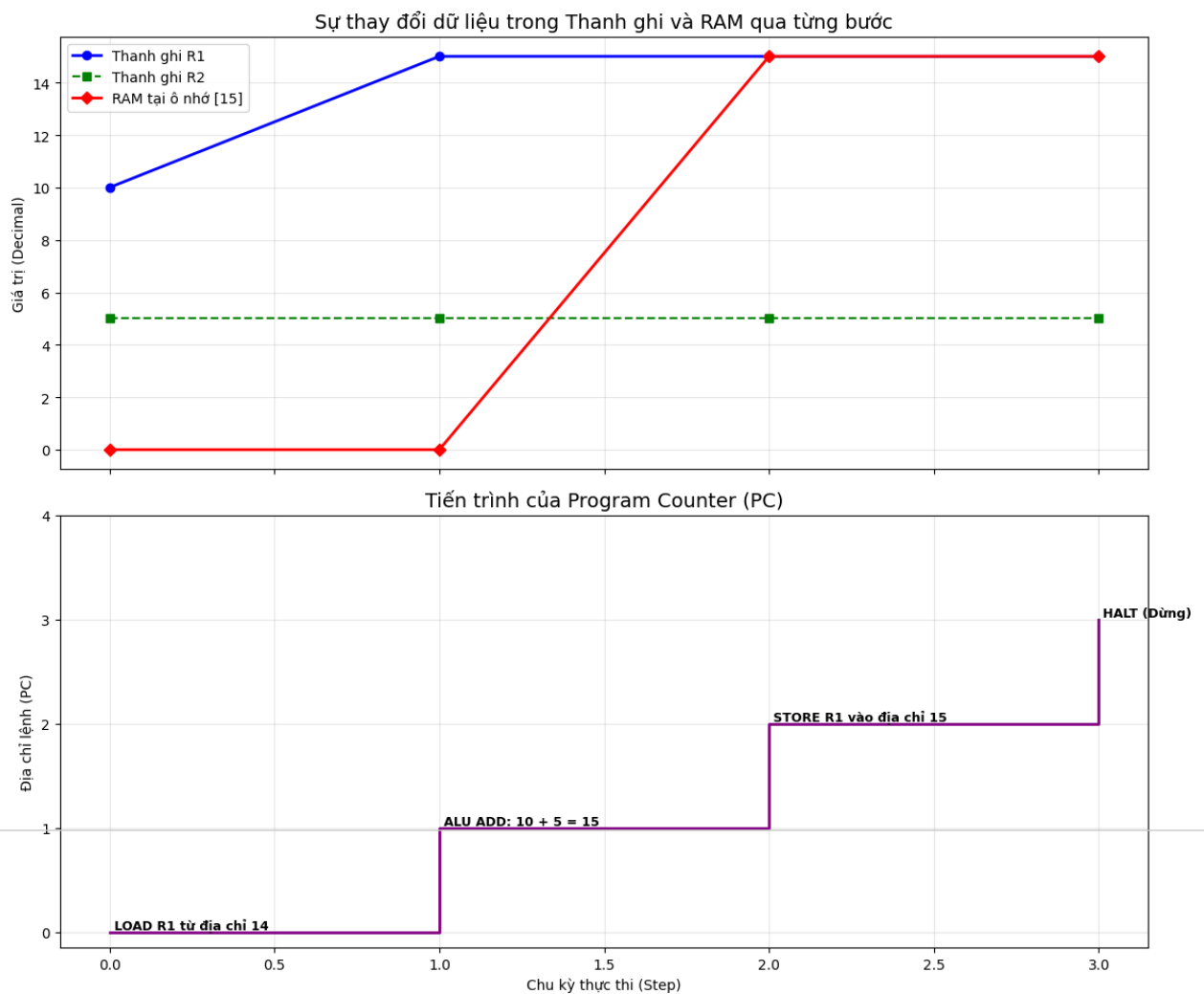
    ax1.set_title("Sự thay đổi dữ liệu trong Thanh ghi và RAM qua từng bước", fontsize=14)
    ax1.set_ylabel("Giá trị (Decimal)")
    ax1.legend()
    ax1.grid(True, alpha=0.3)

    # --- BIỂU ĐỒ 2: TIẾN TRÌNH LỆNH (PC & LỆNH) ---
    # Hiển thị mã lệnh dffới dạng text trên biểu đồ
    for i, row in df.iterrows():
        ax2.text(i, i, f" {row['Hành động']}", verticalalignment='bottom', fontsize=9, fontweight='bold')

    ax2.step(steps, df['PC'], where='post', color='purple', label='Program Counter (PC)', linewidth=2)
    ax2.set_title("Tiến trình của Program Counter (PC)", fontsize=14)
    ax2.set_xlabel("Chu kỳ thực thi (Step)")
    ax2.set_ylabel("Địa chỉ lệnh (PC)")
    ax2.set_yticks(range(max(df['PC']) + 2))
    ax2.grid(True, alpha=0.3)

    plt.tight_layout()
    plt.show()

# Thực thi mô phỏng và vẽ đồ thị
# (Sử dụng lại class SimpleCPU bạn đã cung cấp)
visualize_cpu_execution(history)
```



Dưới đây là bản tổng kết và kết luận hệ thống cho toàn bộ hành trình lập trình mô phỏng từ các linh kiện cơ bản đến bộ vi xử lý hoàn chỉnh mà chúng ta đã thực hiện:

1. Sự hội tụ của Toán học, Logic và Kỹ thuật Toàn bộ các chương trình từ Full Adder, ALU đến CPU minh chứng rằng máy tính không phải là một thực thể huyền bí, mà là sự hiện thực hóa của Đại số tuyến tính và Logic Boolean.

Toán học: Cung cấp thuật toán (như số bù 2 để trừ, thuật toán Booth để nhân).

Logic: Cung cấp công cụ thực thi (các cổng AND, OR, XOR).

Lập trình (Python): Đóng vai trò là môi trường mô phỏng, giúp ta "số hóa" các quy luật vật lý của transistor thành các dòng mã có thể kiểm thử.